

第五讲：预期——转移动态（Transition Dynamics）

异质性主体宏观经济学的计算方法

孙杰

2026 年 5 月 15 日

多伦多大学

HouseholdStages

HouseholdStages

- 一个用于构建异质性主体宏观模型家庭模块的阶段（Stages）实现。
- Auclert–Bardóczy–Rognlie–Straub 的 **SSJ toolkit**（Python）也有阶段实现。
- Econ-ARK 也有某种形式的 Stages。

这个包提供了什么

- **阶段实现。** MarkovStage、WealthChangeStage、ConsumptionSavingsStage、LogitChoiceStage、MigrationStage、BorrowingConstraintStage、AssetPriceChangeStage ...
- **组合 (Composition)。** ○ 把 Stages 串接起来；lift_moments 附加积分。
- **内层求解辅助函数。** 在给定 env (价格与参数) 下求稳态以及转移路径上的 V 、 Λ 。
- **函子化提升 (Functorial lifts)。** lift_jacobian (自动微分)、lift_gpu。
- **SSJ 工具。** 序列空间雅可比 (sequence-space Jacobian) (在 L6+ 中讲解)。

这个包不做什么

这个库处理**给定 env 下的家庭模块**。它不：

- 决定你的总量状态。
- 由 $\bar{K}$ 计算价格（厂商模块）。
- 运行用于校准或市场出清的外循环。

为什么这样切分

- 家庭模块：在所有 HA 模型中**通用的组件**。
- 均衡、校准、转移：**模型特定的**。

稳态内层求解辅助函数

```
# V backward to a fixed point at fixed env.  
solve_vfi_steady_state_given_env!(hh, env)  
  
#  $\Lambda$  forward to stationarity using the policy populated by the backward sweep.  
solve_lambda_steady_state_given_env!(hh)  
  
# Bundle of the two: V then  $\Lambda$ , plus attached moments.  
solve_steady_state_given_env!(hh, env)
```

这三个函数都把 `env` **当作给定**。它们对家庭模块做迭代，但不更新价格或参数。

Aiyagari 稳态的阶段实现

与之前相同的模型

- 阶段链（时间顺序）：

$z_shock \circ_s income \circ_s savings.$

- 厂商。Cobb-Douglas，劳动 L 固定。
- 均衡。 $\bar{K} = \bar{K}^{\text{supplied}}(\bar{K})$ 。
- 外循环。对 $\bar{K}$ 做带阻尼的**试错调整**（tatonnement）。

状态布局

```
layout = StateLayout(  
    StateAxis(:wealth, continuous_grid(0, 100; length=400, spacing=:log)),  
    StateAxis(:z,      [0.6, 1.0, 1.4]),  
)
```

- **wealth** (连续, 400 点对数网格)。
- **z** (生产率; 离散, 3 个水平)。
- 闭包读取 `cell.wealth`、`cell.z`。

阶段 1: z 冲击

```
z_shock = MarkovStage(layout; axis=:z, transition=P_z)
```

阶段 2：收入

```
income = WealthChangeStage(layout;  
    wealth_post = (cell; env) ->  
        (1 + env.r) * cell.wealth + env.w * cell.z,  
    )
```

- 前面的例子把结果对齐到网格上。库函数则做连续的再插值。

阶段 3：消费——储蓄

```
savings = ConsumptionSavingsStage(layout;  $\beta=0.96$ ,  
    utility=(cell, c; env) -> c < 0 ? -Inf : log(c),  
    monotone_search=:divide_conquer,  
)
```

- :divide_conquer: 当策略关于 b 单调时, 使用更快的 $O(N \log N)$ 算法。

组合

```
hh = z_shock ◦ income ◦ savings
```

- ○ 按时间顺序组合 Stages。
- 家庭模块本身就是一个 *Stage*：有自己的 backward! 和 forward!。

定义矩

```
hh = define_moments!(hh;  
    K_supplied=at_end(integrand=:wealth, reduce=sum),  
)
```

- `define_moments!`: 在 Λ 上定义积分。这里 $K^{\text{supplied}} = \int b d\Lambda^{\text{end}}$ 。

在给定 $\bar{K}$ 下求稳态

```
env = make_env(hh; aiyagari_prices(K, p)...)      # (r, w)
res = solve_steady_state_given_env!(hh, env)
K_supplied = res.moments.K_supplied
```

- 第 1 行。构造 env : (K, r, w) 。
- 第 2 行。求稳态的 V 与 Λ 。
- 第 3 行。读出 K^{supplied} 。

外层试错调整循环

```
K = 5.0
for it in 1:max_iter
    env = make_env(hh; aiyagari_prices(K, p)...)
    (; V, Λ, moments) = solve_steady_state_given_env!(hh, env)
    K_supplied = moments.K_supplied
    K_err = abs(K_supplied - K) / K
    K_err <= rtol && break
    K += update_speed * (K_supplied - K)      # damped tatonnement
end
```

每一轮： $\bar{K} \rightarrow R \rightarrow V, \Lambda \rightarrow K^{\text{supplied}} \rightarrow K_{\text{err}}$ ，再更新 $\bar{K}$ 。

当 $|K_{\text{err}}|/K < \text{rtol}$ 时停止。

比较静态

比较静态

改变一个参数，重新求解稳态。

可以把它理解为对冲击的**长期**响应。

在示例校准中：

- $A = 1 \rightarrow \bar{K}_{\text{pre}} = 5.68, \bar{R}_{\text{pre}} = 0.038。$
- $A = 1.05 \rightarrow \bar{K}_{\text{post}} = 6.14, \bar{R}_{\text{post}} = 0.038。$

比较静态

比较静态告诉你冲击的**长期**响应。

但它没告诉你：

- 转移的速度
- 转移的路径
- 冲击发生时在世个体所受的福利影响。

要回答这些问题，就需要转移动态。

转移动态

转移动态

- **稳态。**单一的 (K, V, Λ) 。不随时间变化。
- **转移。**一条序列 $\{(K_t, V_t, \Lambda_t)\}_{t=1, \dots, T}$ 。经济正在运动。

MIT 冲击 (MIT shock)

在 $t = 1$ 时发生的**永久性、未预期到**的冲击。

- $t = 0$: 处于稳态。
- $t = 1$: 参数永久性改变。主体感到意外。
- $t \geq 1$: 对所有外生与内生变量的整条未来路径具有完全预见 (perfect foresight)。

例：TFP 冲击

在 $t = 1$ 时发生的永久性正向 TFP 冲击：

$$A_t = 1.05 \quad \text{for all } t \geq 1, \quad A_0 = 1.$$

- 唯一的外生变化。
- $\{r_t, w_t, K_t, V_t, \Lambda_t\}$ 作为**响应**发生变化。
- 取 $T = 100$ ，并令 $V_{T+1} = V^{\text{new_steady_state}}$ 以闭合模型。

求解 MIT 转移

问题陈述

- 已知: $\{A_t\}_{t=1}^T$ 。
- 求: $\{K_t, V_t, \Lambda_t\}$, 使得
 - $V_{T+1} = V_{ss}$ (终端条件)。
 - $\Lambda_1 = \Lambda_{ss}$ (初始条件)。
 - $V_t = \text{backward}(V_{t+1}, K_t, A_t)$ 。
 - $\Lambda_{t+1} = \text{forward}(\Lambda_t, V_{t+1})$ 。
 - 市场出清: $K_t = K_t^{\text{supplied}}$ 。

算法

1. 猜测路径 $\{K_t\}$ 。

2. 反向迭代 V 。

$$V_{T+1} \leftarrow V_{\text{ssPost}}, \quad V_t = \text{backward}(V_{t+1}, K_t, A_t).$$

3. 正向模拟 Λ 。

$$\Lambda_1 \leftarrow \Lambda_{\text{ssPre}}, \quad \Lambda_{t+1} = \text{forward}(\Lambda_t, V_{t+1}).$$

4. 更新 K_{path} 。

$$K_t \leftarrow (1 - s) K_t + s K_t^{\text{supplied}}.$$

说明：求根

归根到底，问题就是在序列空间中求超额需求函数的根：

$$\text{excess_demand}(\{K_t\}) = \vec{0}.$$

为演示起见，我们使用简单而稳健的试错调整法。

使用序列空间雅可比（sequence-space Jacobian）的梯度方法可以快得多。

算法

```
hh_path = [aiyagari_household(p_new) for _ in 1:T] # one chain per period
K_path = fill(K_ss, T) # initial guess
V_path[T+1] .= V_ss; Λ_path[1] .= Λ_ss # boundary conditions
while res > tol
    env_path = [make_env(hh_path[t]; aiyagari_prices(K_path[t], p_new)...) for t in 1:T]
    for t in T:-1:1 # backward sweep
        copyto!(V_path[t], backward!(hh_path[t], V_path[t+1], env_path[t]))
    end
    for t in 1:T # forward sweep
        copyto!(Λ_path[t+1], forward!(hh_path[t], Λ_path[t]))
        K_supplied[t] = compute_moments(hh_path[t], Λ_path[t+1], env_path[t]).K_supplied
    end
    res = maximum(abs, K_supplied .- K_path)
    K_path .= (1 - update_speed) .* K_path .+ update_speed .* K_supplied # damped tatonnement
end
```

实现

初始稳态

我们需要：

- V_{ss} ：反向扫描的终端值。
- Λ_{ss} ：正向扫描的初始值。
- 第 2 节的试错调整就能同时给出这两者。
- **初始化：** $V_{ss} \rightarrow V_{\text{path}}[T+1]$; $\Lambda_{ss} \rightarrow \Lambda_{\text{path}}[1]$; 初始的 $\{K_t\} \equiv K_{ss}$ 。

反向扫描

```
for t in T:-1:1
    env_t = make_env(hh_path[t]; aiyagari_prices(K_path[t], p_new)...
    copyto!(V_path[t], backward!(hh_path[t], V_path[t+1], env_t))
end
```


正向扫描（注意重新装载 kernels!）

```
for t in 1:T
    env_t = make_env(hh_path[t]; aiyagari_prices(K_path[t], p_new)...
    copyto!(  $\Lambda$ _path[t+1], forward!(hh_path[t],  $\Lambda$ _path[t]))
    K_supplied[t] = compute_moments(hh_path[t],  $\Lambda$ _path[t+1], env_t).K_supplied
end
```

外层试错调整更新

```
res = maximum(abs, K_supplied .- K_path)
K_path .= (1 - update_speed) .* K_path .+ update_speed .* K_supplied
```

脉冲响应：资本

形状。

- ΔA 推高 r 。
- 更高的 $r \rightarrow$ 更多储蓄。
- 更多储蓄 $\rightarrow$ 更高的 $K \rightarrow$ 更低的 r 。具有自我修正性。

滞后。 财富分布有惯性。 K 的峰值比 A 的峰值滞后约 5 期。

脉冲响应： r_t 与 w_t

r_{to}

- 在冲击时点跳升 (A_1 先于 K 变化)。
- 当 K 超调时跌至稳态以下。
- 当 $A_t \rightarrow 1$ 时回到 r_{ss} 。

w_{to}

- 形状相同，符号相反。
- 当 K 超调时高于稳态。

福利

- 富裕主体：在 r 上获得短暂的资本收益。
- 依赖工资的主体：吃到 w 这一侧的好处。

对于 Aiyagari，我们能够逐期得到福利效应的一整个分布。把两条渠道拆开来看，正是 MIT 冲击能让你做的事情之一。

把 T 取得足够大

- $T =$ 使得 $A_T \approx 1$ 且 $V_{T+1} = V_{ss}$ 的视界。
- 取太小 $\rightarrow$ 终端条件会污染整条路径。

经验法则。取 T 使得 $A_T - 1 < 10^{-3}$ 。在 $\rho = 0.85$ 、 $A_0 = 1.05$ 下： $T \approx 25$ 。我们取 $T = 100$ 。